

CMP6207 report context

Household automations on MongoDB replica set **rs0**

Shekhar Lamichhane

30 September 2026

Abstract

This note is the writing brief for the CMP6207 report. The started report is **report.tex**. The talk is **slides.pptx**. The submission is one simulated home, three automations, MQTT, a Node.js rule engine, and MongoDB replica set **rs0**. The report target is about 4,000 words.

1 What was built

One Node.js process serves home **home_001**. Simulated sensors publish on MQTT. The API validates each reading, stores it, updates the latest value, and runs the rules. Commands return on MQTT. The status page and Telegram talk only to the API.

Database	Subsystem	Behaviour
smart_water	tank_roof_01	1000 L roof tank. Warn at 90% with the pump on. Pump off at 98%. Dry-run, pump failure, abnormal flow, and low tank at 15%.
smart_focus	focus_desk_01	Desk presence. Lamp at 6500 K and 100% brightness while someone is sitting. Lamp off when the desk is empty.
smart_climate	climate_living_01	Cooling starts above 28°C and stops at 26°C.

All three databases use the same replica set **rs0** on ports 27017, 27018, and 27019. The set is already initialised. Telemetry and **device_latest** use write concern **w:1**. Incidents use **w:"majority"**. The rule engine reads from the primary. **smart_water** also keeps **tanks**. Water documents still carry **tank_id**. Every automation document carries **subsystem_id**.

2 Facts that stay fixed

- Telemetry topic:
`ioThings/{home}/{subsystem}/{device_type}/{device_id}/telemetry`
- Command topic: the same path ending in `/command`, at QoS 1.
- Water devices are level, flow, pump state, and inlet presence. The inlet sensor separates a dry pump from a broken pump.
- Litres in the tank are calculated when the tank is read. They are not stored as their own sensor message.
- Litres saved on cutoff is flow in litres per minute times 20. The overflow scenario uses 15 L/min, so the recorded save is 300 L.
- A multi-sensor rule runs only when the readings it needs were updated together.

- The engine does not open a second incident for a rule that already has an open incident.
- Telegram is sent only when the bot token is the full BotFather value and the chat id is the person's own id. The API reads `.env` only at startup.
- A phone app, real hardware, a pet feeder, pump schedules, and starting the pump on a low tank were left out on purpose.

3 Word budget

Section	Words	Job
Introduction	400	The household problem and what was built.
NoSQL appraisal	700	Document model against a relational schema for this workload.
Architecture	500	MQTT, the API, the rule engine, and the status page.
Replica set	600	<code>rs0</code> , write concern, and the failover test still to record.
Data model and rules	700	Collections and the water, desk, and climate rules.
Demonstration	700	Scenarios, notifications, and observed results.
Evaluation and conclusion	400	What the design is good at, and what is still simulated.
Total	4000	

4 Evidence still to save

Put screenshots in `evidence/` before the final PDF. Do not invent the figures.

- `rs.status()` with one primary and two secondaries.
- One document from each collection.
- Overflow: a warning, then the pump off at 98%.
- Dry-run, pump failure, abnormal flow, and a confirmed fault after 30 seconds.
- One `study` notification and one `cool` notification.
- A notification that was delivered on Telegram.
- Failover: stop the primary during an overflow, record the election, show the cutoff incident still stored, then bring the node back.

5 How to run a case while writing

From the project root, with the replica set already up: `npm run seed`, then `npm run dev`. In a second terminal run one of `npm run simulate -- overflow`, `study`, or `cool`. `cool` changes only the living room. The tank and the desk stay on their last stored readings until their own simulators run. Stop `cool` with Ctrl+C so cooling alerts do not repeat.